

Q1: (50 points) Consider the inventory control problem in Lecture 2:

$$w_t = \begin{cases} 1 & \text{with prob. } 1/2 \\ 3 & \text{with prob. } 1/2 \end{cases}$$

$$c(x) = \begin{cases} x & \text{if } x \geq 0 \\ -1000x & \text{if } x < 0. \end{cases}$$

and $g_t(x_t, u_t) = c(x_t) + 2u_t$. $g_N(x_N) = c(x_N)$. Consider $N = 2$.

In Lecture 2, we have shown that $\mu_1^*(x) = 3 - x$ and $J_1^*(x) = c(x) - 2x + 7$. Please continue the DP procedure and verify that $\mu_0^*(x) = 3 - x$ and $J_0^*(0) = 12$.

Instructions for Q2 and Q3:

- You only need to **choose one of the two questions to solve**.
- **Q2 is a coding exercise and Q3 is a theoretical question**.
- You are **welcome to solve both questions**. If you solve both Q2 and Q3, the points earned from both questions will be added together
- If the total points of HW1 are higher than 100, the additional points will be counted as **bonus points**.
- Even though the problem description is long for Q2 and Q3, the actual questions for you to solve are quite approachable.
- If you encounter any difficulties when solving HW1, please contact the instructor.

Q2: GridWorld (50 points) *Coding Exercise.* In this homework, you will study a robot moving in a stochastic two-dimensional environment. You will first solve a finite-horizon Markov decision process (MDP) assuming that the robot's state is fully observable. Then, you will consider the partially observable case, where the robot receives noisy measurements of its location and must maintain a belief over possible states. Finally, you will compare two approximate control strategies under partial observation: a maximum-a-posteriori (MAP) policy and a QMDP policy.

The following Python files are provided:

- `Basic_Functions.py`: defines the grid world, state and action spaces, transition probabilities, observation probabilities, cost functions, and simulation utilities. You **don't** need to edit this file.
- `Belief_Visualization.py`: provided code for visualizing the evolution of the belief state. You **don't** need to edit this file.
- `Policy_Performance_Table.py`: provided code for comparing MAP and QMDP through repeated simulations. You **don't** need to edit this file.
- `MDP.py`: contains the finite-horizon dynamic programming problem. **You will complete** the backward dynamic programming portion of this file.
- `POMDP.py`: contains the belief-state update. **You will complete** the exact Bayesian belief update.
- `MAP.py`: **You will complete** the implementation of the MAP policy.
- `QMDP.py`: contains the implementation of the QMDP policy. You **don't** need to edit this file.

You are not expected to understand every implementation detail in the provided files. However, you should read the comments in `MDP.py` and `POMDP.py` to understand the structure of the state space, action space, transition probability, stage cost, terminal cost, and the functions used to simulate the next state and observation.

Problem Settings. Consider a 6×6 stochastic gridworld in Figure 1. The goal state is $G = (5, 5)$, and is absorbing, i.e. once the robot hits the goal, the agent will stay there with probability 1. The obstacle cells are $(2, 2)$, $(2, 3)$, $(3, 3)$, which cannot be entered. Therefore, there are 33 states in total and the state space is $\{0, 1, \dots, 32\}$ (the Python code uses zero-based indexing.)

The available actions are $\mathcal{A} = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$. When an action is selected, the robot moves in the intended direction with probability 0.8. With probability 0.1 each, it slips to one of the two directions orthogonal to the intended direction. If a movement would leave the grid or enter an obstacle, the robot remains in its current state. The transition probabilities are provided in `Basic_Functions.py`.

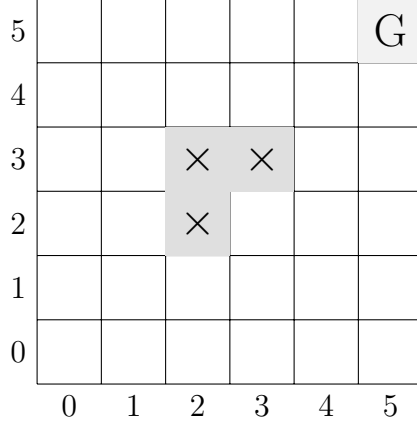


Figure 1: The 6×6 gridworld. G denotes the absorbing goal state and $\times$ denotes an obstacle.

Consider a finite horizon $N = 15$. The objective of the robot is to reach the goal state as quickly as possible. The stage cost is defined as

$$c(x, u) = \begin{cases} 0, & x = x_G, \\ 1, & x \neq x_G, \end{cases}$$

where $x_G = 32$ is the index of the goal state $(5, 5)$. Thus, the robot incurs a cost of one unit at every time step before reaching the goal. Once the goal is reached, the stage cost is zero. The stage cost does not depend explicitly on the selected action.

The terminal cost at the finite horizon N is

$$c_N(x) = \begin{cases} 0, & x = x_G, \\ 15, & x \neq x_G. \end{cases}$$

Therefore, if the robot has not reached the goal by the end of the planning horizon, it receives an additional terminal penalty of 15.

- 1) (15 points) For this subquestion, assume that the robot observes its state x_t exactly.
 - (1a) Complete the dynamic programming section in `MDP.py`. Your program should calculate $J_t^*(x)$ for every valid state x and every $t = 0, \dots, N$. The dynamic programming algorithm is $J_N^*(x) = c_N(x)$, and

$$J_t^*(x) = \min_{u \in \mathcal{U}} \left\{ c(x, u) + \sum_{x'} P(x' | x, u) J_{t+1}^*(x') \right\}, \quad t = N - 1, \dots, 0.$$

- (1b) Provide the heat map visualization of J_0^* and J_6^* by running `MDP.py`.
 - (1c) For the same location x , what is generally larger, J_0^* or J_6^* ? Can you provide an intuitive explanation for this phenomenon?

- 2) (15 points) Now suppose that the true state is not directly observable. Instead, the robot receives a noisy observation y_t . The observation model $P(y_t | x_t)$ is provided in `Basic_Functions.py`.

The belief state is defined as

$$b_t(x) = P(x_t = x | y_0, \dots, y_t, u_0, \dots, u_{t-1}).$$

After action u_t and new observation y_{t+1} , the belief update is

$$b_{t+1}(x') = \frac{P(y_{t+1} | x') \sum_x P(x' | x, u_t) b_t(x)}{\sum_{\tilde{x}} P(y_{t+1} | \tilde{x}) \sum_x P(\tilde{x} | x, u_t) b_t(x)} = \frac{\text{Term 1}(x')}{\sum_{\tilde{x}} \text{Term 1}(\tilde{x})}$$

- (2a) Complete the function `bayes_update()` in `POMDP.py`.
- (2b) Run `Belief_Visualization.py`. Provide the heat map visualization of belief states generated by `Belief_Visualization.py`.
- 3) (10 points) A simple way to make decisions under partial observation is to first identify the most likely state,

$$\hat{x}_t = \arg \max_x b_t(x),$$

and then treat $\hat{x}_t$ as if it were the true state, i.e.

$$\mu_t^{\text{map}}(b_t) = \arg \min_u \left[c(\hat{x}_t, u) + \sum_{x'} P(x' | \hat{x}_t, u) J_{t+1}^*(x') \right]$$

where J_{t+1}^* is obtained from the fully observable case and has been computed in Question (1a). Complete the function `choose_action_MAP()` in `MAP.py`.

- 4) (5 points) The QMDP policy uses the complete belief distribution rather than only its most likely state. For each candidate action u , it evaluates

$$\sum_x b_t(x) \left[c(x, u) + \sum_{x'} P(x' | x, u) J_{t+1}^*(x') \right],$$

and selects the action with the smallest belief-weighted expected cost. The implementation is provided in `QMDP.py`. Read the function `choose_action_QMDP()`.

- 5) (5 points) Run `Policy_Performance_Table.py`. The script simulates repeated episodes using the same transition model, observation model, Bayesian belief update, finite horizon, and cost functions for the two policies.

Report the following quantities in a table for both MAP and QMDP:

- success rate,
- average cumulative cost,

- average time to reach the goal among successful runs.

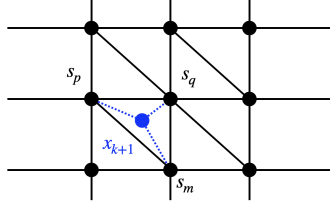
Which policy performs better according to each of the three performance measures?
Do you have intuitive explanations for your observation?

Submission Please submit:

1. All code files in a zip file.
2. A report consisting of answers to all conceptual questions and the requested figures and numerical results.

Only edit the files `MDP.py`, `POMDP.py`, `MAP.py`. Do not modify any of the other provided files

Use the provided random seed when generating the results reported in your submission so that your results are reproducible.

Q3: Discretization of Convex Space (50 points) *Theoretical Question.*

Consider a finite horizon N . Consider a problem with state space S , where S is a convex set of $\mathbb{R}^n$. Suppose that $\hat{S} = \{s_1, \dots, s_M\}$ is a finite subset of S such that S is the convex hull of $\hat{S}$, and consider a one-step lookahead policy based on approximate cost-to-go functions $\tilde{J}_0, \dots, \tilde{J}_N$ defined as follows:

$$\tilde{J}_N(x) = g_N(x), \forall x \in S$$

where $g_N(x)$ is real-valued and convex over S . For $k = 0, \dots, N-1$,

$$\tilde{J}_k(x) = \min \left\{ \sum_{i=1}^M \lambda_i \hat{J}_k(s_i) \mid \sum_{i=1}^M \lambda_i s_i = x, \sum_{i=1}^M \lambda_i = 1, \lambda_i \geq 0, i = 1, \dots, M \right\},$$

where $\hat{J}_k$ is defined by

$$\hat{J}_k(x) = \min_{u \in \mathbb{U}_k(x)} \mathbb{E}(g_k(x, u, w_k) + \tilde{J}_{k+1}(f_k(x, u, w_k))), \forall x \in S \quad (1)$$

even though the algorithm implementation only requires the computation of $\hat{J}_k(y_i)$ for $i = 1, \dots, M$. Let μ_k denote the optimal policy solving the equality (1).

Thus, $\tilde{J}_k$ is obtained from $\tilde{J}_{k+1}$ as a grid-based convex piecewise linear approximation to $\hat{J}_k$ based on the M values:

$$\hat{J}_k(s_1), \dots, \hat{J}_k(s_M).$$

Assume that the cost functions g_k and the system functions f_k are such that the function $\hat{J}_k$ is real-valued and convex over S whenever $\tilde{J}_{k+1}$ is real-valued and convex over S .

Let $J_k(x)$ denote the cost-to-go starting from x at stage k using policy sequence $\vec{\mu}$. By dynamic programming, we have

$$J_k(x) = \mathbb{E}(g_k(x, \mu_k(x), w_k) + J_{k+1}(f_k(x, \mu_k(x), w_k))), \forall x \in S. \quad (2)$$

- (a) (15 points) Prove that $\hat{J}_k(x) \leq \tilde{J}_k(x)$ for all $x \in S, k = 0, 1, \dots, N-1$. (Hint: use convexity)
- (b) (20 points) Prove that $J_{N-1}(x) = \hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x)$ for all $x \in S$, where we consider $J_N(x) = \tilde{J}_N(x) = \hat{J}_N(x) = g_N(x)$. (Hint: use the definitions of J_k and $\hat{J}_k$ and question (a) and (2))
- (c) (15 points) Prove that $J_{N-2}(x) \leq \hat{J}_{N-2}(x) \leq \tilde{J}_{N-2}(x)$ for all $x \in S$. (Hint: use the definitions of $J_k, \hat{J}_k$ and (a)-(b) and (2))

- (d) (**Optional Bonus Question (20 points)**) Try to finish the proof of $J_k(x) \leq \hat{J}_k(x) \leq \tilde{J}_k(x)$ for all $x \in S$ and all $k = 0, 1, \dots, N - 1$.

Remark: this problem provides theoretical justification of a rollout version of dynamic programming with Barycentric Interpolation.